Python Programs

1) import matplotlib.pyplot as plt

```
# Sample data
x = [0, 1, 2, 3, 4, 5]
y1 = [0, 0.5, 0.8, 0.3, -0.2, -0.7]
y2 = [1, 0.9, 0.1, -0.6, -1, -0.7]

# Line plot with custom colors and styles
plt.plot(x, y1, color='blue', linestyle='--', label='Data 1')  # Dashed line
plt.plot(x, y2, color='red', linestyle='-', label='Data 2')   # Solid line

# Labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Line Plot with Custom Colors and Styles')

# Add legend
plt.legend()

# Show plot
plt.show()
```

2) import matplotlib.pyplot as plt

```
# Sample data
```

```
x = [0, 1, 2, 3, 4, 5]
y = [0, 0.5, 0.8, 0.3, -0.2, -0.7]

# Line plot
plt.plot(x, y, color='green')

# Set axes limits
plt.xlim(1, 4)  # Limit x-axis
plt.ylim(-1, 1)  # Limit y-axis

# Labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Line Plot with Adjusted Axes Limits')

# Show plot
plt.show()
```

3) import matplotlib.pyplot as plt

```
# Sample data
x = [0, 1, 2, 3, 4, 5]
y = [0, 0.5, 0.8, 0.3, -0.2, -0.7]

# Line plot
plt.plot(x, y, color='purple', label='Data')

# Labels and title
plt.xlabel('Time (s)', fontsize=12)
```

```
plt.ylabel('Value', fontsize=12)
plt.title('Labelled Line Plot', fontsize=14)

# Add legend
plt.legend()

# Show plot
plt.show()
```

4) import matplotlib.pyplot as plt

```
# Sample data with meaningful context
x = [0, 1, 2, 3, 4, 5]  # Time (weeks)
y = [0, 0.5, 0.8, 0.3, -0.2, -0.7]  # Test scores (normalized, -1 to 1)
sizes = [100, 200, 50, 300, 150, 250]  # Study hours (scaled for visualization)
colors = ['red', 'blue', 'green', 'purple', 'orange', 'cyan']  # Subjects
subjects = ['Math', 'Science', 'History', 'English', 'Art', 'Music']  # Subject labels

# Create scatter plot
plt.scatter(x, y, s=sizes, c=colors, marker='o', alpha=0.6, label='Student Scores')

# Add labels and title
plt.xlabel('Time (Weeks)', fontsize=12)
plt.ylabel('Normalized Test Score', fontsize=12)
plt.title('Student Test Scores Over Time by Subject', fontsize=14)

# Add grid for better readability
plt.grid(True, linestyle='--', alpha=0.7)
```

```
# Add legend
plt.legend(loc='upper right')

# Show plot
plt.show()
```

5) import matplotlib.pyplot as plt

```
# Sample data: positive test scores (out of 100) for three subjects
data = [
    [85, 90, 78, 92, 88, 80],  # Math scores
    [75, 82, 88, 79, 85, 77],  # Science scores
    [90, 87, 83, 91, 86, 89]   # History scores
]
subjects = ['Math', 'Science', 'History']  # Subject labels
colors = ['lightblue', 'lightgreen', 'lightcoral']  # Custom colors for boxes

# Create box plot
boxes = plt.boxplot(data, labels=subjects, patch_artist=True)

# Customize box colors
for patch, color in zip(boxes['boxes'], colors):
    patch.set_facecolor(color)

# Labels and title
plt.xlabel('Subjects', fontsize=12)
plt.ylabel('Test Scores (out of 100)', fontsize=12)
plt.title('Test Score Distribution by Subject', fontsize=14)
```

```
# Add grid for readability
plt.grid(True, linestyle='--', alpha=0.7, axis='y')

# Add annotation for median of Math (first subject)
math_median = sorted(data[0])[len(data[0])//2]  # Approximate median
plt.annotate(f'Math Median: {math_median}',
             xy=(1, math_median),  # Position at first box
             xytext=(1.5, math_median + 5),
             arrowprops=dict(facecolor='black', shrink=0.05))

# Add legend for box colors
for i, subject in enumerate(subjects):
    plt.plot([], [], color=colors[i], label=subject, linewidth=10)  # Dummy plot for legend
plt.legend(title='Subjects', loc='upper right', fontsize=10)

# Show plot
plt.show()
```

6) NumPy is a Python library for efficient array operations, especially for numerical and scientific computing. It supports multi-dimensional arrays and provides fast mathematical functions for data analysis and machine learning.

Program: Create a 2D NumPy array, add 5 to each element, and compute the sum of each row.

```
python
import numpy as np

# Create a 2D NumPy array
array_2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:\n", array_2d)
```

```
# Add 5 to each element
added = array_2d + 5
print("Array after adding 5:\n", added)

# Compute sum of each row
row_sums = np.sum(array_2d, axis=1)
print("Row Sums:", row_sums)
```

7) import numpy as np

```
# Create arrays
zeros_array = np.zeros((2, 3))
ones_array = np.ones((2, 3))
range_array = np.arange(1, 7).reshape(2, 3)
linspace_array = np.linspace(0, 10, 6).reshape(2, 3)

print("Zeros Array:\n", zeros_array)
print("Ones Array:\n", ones_array)
print("Range Array:\n", range_array)
print("Linspace Array:\n", linspace_array)

# Combine arrays
combined = np.vstack((range_array, linspace_array))
print("Combined Array (Vertical Stack):\n", combined)
```

8) mport numpy as np

```
# Create a 3D array
array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]], [[9, 10], [11, 12]]])
print("3D Array:\n", array_3d)

# Extract specific element
element = array_3d[1, 0, 1]
print("Element at [1, 0, 1]:", element)

# Slice a subarray
subarray = array_3d[0:2, :, 1]
print("Sliced Subarray (last column of first two layers):\n", subarray)

# Modify a slice
array_3d[2, :, :] = 0
print("Modified 3D Array:\n", array_3d)
```

9) import numpy as np

```
# Create a 2D array
array = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print("Original Array:\n", array)

# Fancy indexing with index arrays
rows = [0, 2]
cols = [1, 2]
selected = array[rows, cols]
print("Selected Elements [0,1], [2,2]:", selected)

# Boolean indexing for elements > 50
mask = array > 50
```

```
array[mask] = 0
print("Array after setting elements > 50 to 0:\n", array)
```

10)
```
import numpy as np

# Create a 2D array
array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Original Array:\n", array)

# Element-wise operations
squared = array ** 2
added = array + 10
print("Squared Array:\n", squared)
print("Array + 10:\n", added)

# Statistical operations
mean = np.mean(array)
std_dev = np.std(array)
print("Mean:", mean)
print("Standard Deviation:", std_dev)
```

11)
```
import numpy as np

# Create an array
angles = np.array([0, np.pi/4, np.pi/2])
print("Angles (radians):", angles)

# Trigonometric functions
sines = np.sin(angles)
```

```
cosines = np.cos(angles)
print("Sines:", sines)
print("Cosines:", cosines)

# Exponential and logarithmic functions
values = np.array([1, 2, 3])
exp_values = np.exp(values)
log_values = np.log(values)
print("Exponential:", exp_values)
print("Natural Log:", log_values)
```

12)
```
import numpy as np

# Simulated dataset
data = np.array([10, 20, 15, 30, 25, 5, 35, 40])
print("Original Data:", data)

# Normalize data (scale to [0, 1])
normalized = (data - np.min(data)) / (np.max(data) - np.min(data))
print("Normalized Data:", normalized)

# Filter values greater than mean
mean = np.mean(data)
filtered = data[data > mean]
print("Values > Mean (", mean, "):", filtered)
```

13)
```
import numpy as np

# Create an array
```

```
array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Original Array:\n", array)

# Save to text file
np.savetxt('array.txt', array, fmt='%d', delimiter=',')

# Load from text file
loaded_array = np.loadtxt('array.txt', delimiter=',')
print("Loaded Array:\n", loaded_array)
```

14) import numpy as np

```
# Define a 3x3 matrix and a vector
A = np.array([[4, 2, 1], [1, 3, 0], [2, 1, 5]])
b = np.array([8, 3, 11])

# Solve Ax = b
x = np.linalg.solve(A, b)
print("Solution to Ax = b:", x)

# Compute determinant and inverse
det = np.linalg.det(A)
inv_A = np.linalg.inv(A)
print("Determinant of A:", det)
print("Inverse of A:\n", inv_A)

# Verify solution
print("Verification (A @ x):", np.dot(A, x))
```

15) import numpy as np

```
# Set seed for reproducibility
np.random.seed(42)

# Generate random arrays
uniform = np.random.uniform(0, 10, size=(2, 3))
normal = np.random.normal(loc=5, scale=2, size=(2, 3))

print("Uniform Random Array:\n", uniform)
print("Normal Random Array:\n", normal)

# Statistical analysis
uniform_mean = np.mean(uniform)
normal_std = np.std(normal)
print("Mean of Uniform Array:", uniform_mean)
print("Standard Deviation of Normal Array:", normal_std)

# Generate random integers
random_ints = np.random.randint(1, 100, size=5)
print("Random Integers:", random_ints)
```